

Chapter 3

Arrays and Strings

Array

Array is the collection of similar data types or collection of similar entity stored in contiguous memory location.

- ✓ Array of character is called string.
- ✓ Each data item of an array is called an element
- ✓ Each element is unique and located in separated memory location.
- ✓ Each of elements of an array share a variable but each element having different index no.
- ✓ The length and type of an array cannot be changed after array declaration.

Types of an Array

An array can be a three type, that is

- 1) Single Dimensional
- 2) Two Dimensional
- 3) Multi-Dimensional

Single Dimensional Array

Single-dimensional arrays are the simplest form of arrays. These types of arrays are used to store number of items of a predefined type. All items in a single dimension array are stored in a row starting from 0 to the size of array.

Declaration of Single Dimensional Array

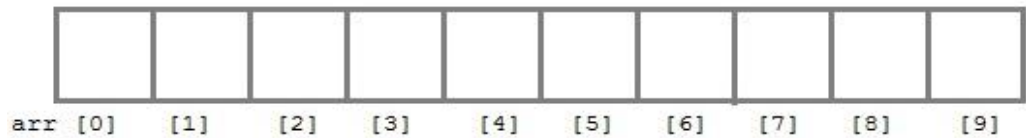
The declaration of an array tells the compiler that, the data type, name of the array, size of the array and for each element it occupies memory space.

Syntax

data_type array_name [size];

Here, data types are any pre-defined type and array name and size can be user desire. For example

```
int arr[10];
```



Here *int* is the data type, *arr* is the name of the array and 10 is the size of array. It means array *arr* can only contain 10 elements of *int* type. Index of an array starts from 0 to size-1 i.e first element of *arr* array will be stored at *arr[0]* address and last element will occupy *arr[9]*.

Like for *int* data type, it occupies 2 bytes for each element and for float it occupies 4 byte for each element etc. The size of the array operates the number of elements that can be stored in an array and it may be an *int* constant or constant *int* expression.

Initialization of an Array

After an array is declared it must be initialized. Otherwise, it will contain garbage value (any random value). An array can be initialized at either compile time or at runtime. The compile time initialization can categories into 4, that is

- (i) Initializing all specified memory locations.
- (ii) Partial array initialization
- (iii) Initialization without size.
- (iv) String initialization.

Compile time Array initialization

We can initialize the elements of arrays in the same way as the ordinary variables when they are declared. The general form of initialization of arrays is

Syntax

```
data_type array-name[size] = { list of values };
```

Example

```
int marks[4]={ 67, 87, 56, 77 }; //integer array initialization

int marks[]={10,20,30};

char ch[3]={ 'a','b','c' };      //float array initialization

float area[5]={ 23.4, 6.8, 5.5 }; //float array initialization
```

(i) Initializing all specified memory locations: Arrays can be initialized at the time of declaration when their initial values are known in advance. Array elements can be initialized with data items of type *int*, *char* etc.

```
int Arr[5]={10,15,1,3,20};
```

During compilation, 5 contiguous memory locations are reserved by the compiler for the variable Arr and all these locations are initialized as shown in figure.

Arr[0]	Arr[1]	Arr[2]	Arr[3]	Arr[4]
10	15	1	3	20
1000	1002	1004	1006	1008

(ii) Partial array initialization: Partial array initialization is possible in C language. If the number of values to be initialized is less than the size of the array, then the elements will be initialized to zero automatically.

```
int a[5]={10,15};
```

Even though compiler allocates 5 memory locations, using this declaration statement; the compiler initializes first two locations with 10 and 15, the next set of memory locations are automatically initialized to 0's by compiler as shown in figure.

Arr[0]	Arr[1]	Arr[2]	Arr[3]	Arr[4]
10	15	0	0	0
1000	1002	1004	1006	1008

Initialization with all zeros: these kinds of arrays are initialized with all zeros.

```
int a[5]={0};
```

Arr[0]	Arr[1]	Arr[2]	Arr[3]	Arr[4]
0	0	0	0	0
1000	1002	1004	1006	1008

(iii) Initialization without size: Consider the declaration along with the initialization.

```
int ch[]={1,0,3,5} // array size is 4
char b[]={'C','O','M','P','U','T','E','R'};
```

In this declaration, even though we have not specified exact number of elements to be used in array b, the array size will be set of the total number of initial values specified. So, the array size will be set to 8 automatically. The array b is initialized as shown in figure.

b[0]	b[1]	b[2]	b[3]	b[4]	b[5]	b[6]	b[7]
C	O	M	P	U	T	E	R
1000	1001	1002	1003	1004	1005	1006	1007

(iv) Array initialization with a string: -Consider the declaration with string initialization.

```
char b[]="COMPUTER";
```

The array b is initialized as shown in figure.

b[0]	b[1]	b[2]	b[3]	b[4]	b[5]	b[6]	b[7]	b[8]
C	O	M	P	U	T	E	R	\0
1000	1001	1002	1003	1004	1005	1006	1007	1008

Even though the string "COMPUTER" contains 8 characters, because it is a string, it always ends with null character. So, the array size is 9 bytes (i.e., string length 1 byte for null character).

```
char b[9]="COMPUTER"; // correct
char b[8]="COMPUTER"; // wrong
```

Program xx: Compile time array initialization

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
{
    int i;
    int arr[]={2,3,4};
    for(i=0 ; i<3 ; i++) {
        printf("%d\t",arr[i]);
    }
    getch();
}
```

Output

2 3 4

One important thing to remember is that when you will give more initializer than declared array size than the compiler will give an error. For example

```
int marks[4]={ 67, 87, 56, 77, 59 }; //Compile time error
```

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large array, or to initialize array with user specified values. Example,

```
void main()
{
    int arr[4];
    int i, j;
    printf("Enter array element");
    for(i=0;i<4;i++)
    {
        scanf("%d",&arr[i]); //Run time array initialization
    }
}
```

```

for(j=0;j<4;j++)
{
    printf("%d\n",arr[j]);
}
getch();
}

```

Accessing Array Elements

An array element is accessed by writing the name of the array followed by the subscript (index) in square brackets. For example

```

int arr[4]={10,20,30,40};

int salary = arr[3];

```

The above example will take 4th element from the array and assign the value to salary variable.

Two Dimensional Arrays

An array consisting of two subscripts is known as two-dimensional array. These are often known as array of the array. In two dimensional arrays the array is divided into rows and columns. These are well suited to handle a table of data. In 2-D array we can declare an array as:

Syntax:

```

data_type array_name[row_size][column_size];

```

Ex:- int arr[3][3];

where first index value shows the number of the rows and second index value shows the number of the columns in the array.

Initializing two-dimensional arrays

Like the one-dimensional arrays, two-dimensional arrays may be initialized by following their declaration with a list of initial values enclosed in braces.

Ex: `int a[2][3]={0,0,0,1,1,1};`

initializes the elements of the first row to zero and the second row to one. The initialization is done row by row. The above statement can also be written as

```
int a[2][3] = {{ 0,0,0},{1,1,1}};
```

by surrounding the elements of each row by braces. We can also initialize a two-dimensional array in the form of a matrix as shown below

```
int a[2][3]={
    {0,0,0},
    {1,1,1}
};
```

When the array is completely initialized with all values, explicitly we need not specify the size of the first dimension.

```
Ex: int a[][3]={
    {0,2,3},
    {2,1,2}
};
```

If the values are missing in an initializer, they are automatically set to zero.

```
Ex: int a[2][3]={
    {1,1},
    {2}
};
```

Will initialize the first two elements of the first row to one, the first element of the second row to two and all other elements to zero.

The arrays are stored in the memory in terms of the row design, i.e. first the first row of the array is stored in the memory then second and so on. Suppose we have an array named `arr` having 3 rows and 3 columns then it can be stored in the memory in the following manner.

```
int arr[3][3];
```


arr[0][0]	arr[0][1]	arr[0][2]
arr[1][0]	arr[1][1]	arr[1][2]
arr[2][0]	arr[2][1]	arr[2][2]

However the actual representation of this array in memory would be something like

Store Data into 2D array at runtime

...

```
int abc[5][4];
```

.....

```
/*loop for first dimension which is 5 here*/
```

```
for(i=0; i<=4; i++)
```

```
{
```

```
    /*loop for second dimension of array which is 4 in this example*/
```

```
    for(j=0;j<=3;j++)
```

```
    {
```

```
        printf("Enter value for abc[%d][%d]:", i, j);
```

```
        scanf("%d", &abc[i][j]);
```

```
    }
```

```
}
```

arr[0][1]	arr[0][2]	arr[0][3]	arr[1][0]	arr[1][1]	-----	arr[4][2]	arr[4][3]
8002	8004	8006	8008	8010		8024	8026

Multidimensional Arrays

Multidimensional arrays are often known as array of the arrays. In multidimensional arrays the array is divided into rows and columns, mainly

while considering multidimensional arrays we will be discussing mainly about two dimensional arrays and a bit about three dimensional arrays.

Syntax:

```
data_type array_name[size1][size2][size3]-----[sizeN];
```

In 2-D array we can declare an array as:

```
int arr[3][3] = { 1, 2, 3,
                  4, 5, 6,
                  7, 8, 9 };
```

where first index value shows the number of the rows and second index value shows the number of the columns in the array. To access the various elements in 2-D array we can use: `printf("%d", a[2][3]);`

In 3-D we can declare the array in the following manner :

```
int arr[3][3][3] = { 1, 2, 3,
                     4, 5, 6,
                     7, 8, 9,
                     10, 11, 12,
                     13, 14, 15,
                     16, 17, 18,
                     19, 20, 21,
                     22, 23, 24,
                     25, 26, 27
                     };
```

If we want to access the element the in 3-D array we can do it as follows:

```
printf("%d",a[2][2][2]);
```

Program xx: Multi-dimensional arrays

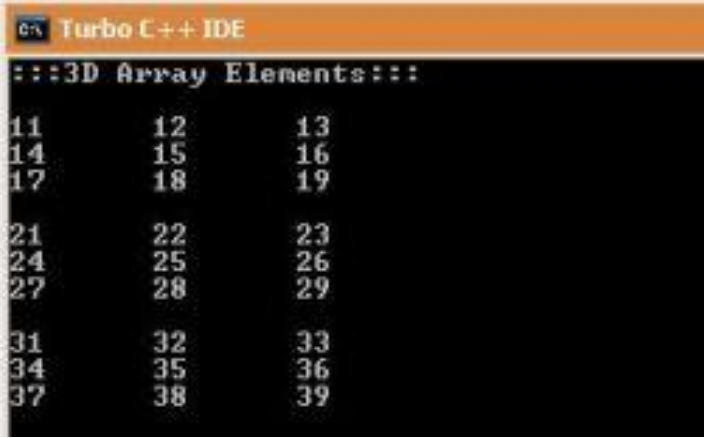
```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
{
int i, j, k;
int arr[3][3][3]=
    {
        {
            {11, 12, 13},
            {14, 15, 16},
            {17, 18, 19}
        },
        {
            {21, 22, 23},
            {24, 25, 26},
            {27, 28, 29}
        },
        {
            {31, 32, 33},
            {34, 35, 36},
            {37, 38, 39}
        },
    };
clrscr();
printf(":::3D Array Elements:::\n\n");
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        for(k=0;k<3;k++)
        {
```

```
    printf("%d\t",arr[i][j][k]);  
    }  
    printf("\n");  
}  
printf("\n");  
}  
getch();  
}
```

Output



The screenshot shows the Turbo C++ IDE's output window. The title bar reads "Turbo C++ IDE". The output text is as follows:

```
:::3D Array Elements:::  
  
11      12      13  
14      15      16  
17      18      19  
  
21      22      23  
24      25      26  
27      28      29  
  
31      32      33  
34      35      36  
37      38      39
```

Strings

In C language a string is group of characters (or) array of characters, which is terminated by delimiter \0 (null). Thus, C uses variable-length delimited strings in programs.

Declaring Strings

does not support string as a data type. It allows us to represent strings as character arrays. In C, a string variable is any valid C variable name and is always declared as an array of characters.

Syntax

```
char string_name[size];
```

The size determines the number of characters in the string name.

Example:

```
char city[10];
char name[30];
```

Initializing strings

There are several methods to initialize values for string variables.

```
char str[6]= "Hello";
```

H	E	L	L	O	\0
---	---	---	---	---	----

```
char month[] = "JANUARY"
```

J	A	N	U	A	R	Y	\0
---	---	---	---	---	---	---	----

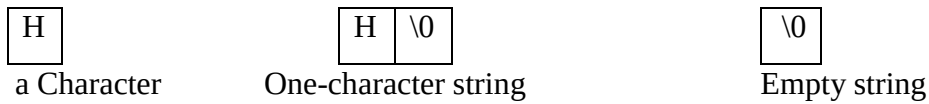
```
char city[8]={ 'N', 'E', 'W', 'Y', 'O', 'R', 'K', '\0' };
```

The string city size is 8 but it contains 7 characters and one character space is for NULL terminator.

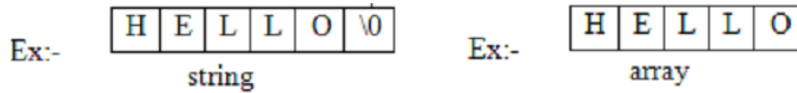
Storing strings in memory

In C a string is stored in an array of characters and terminated by \0 (null). A string is stored in array, the name of the string is a pointer to the beginning of the

string. The character requires only one memory location. If we use one-character string it requires two locations. The difference is shown below,



The difference between array and string is shown below,



Because strings are variable-length structure, we must provide enough room for maximum-length string to store and one byte for delimiter.

Why do we need null?

A string is not a data type but a data structure. String implementation is logical not physical. The physical structure is array in which the string is stored. The string is variable-length, so we need to identify logical end of data in that physical structure.

String constant (or) Literal

String constant is a sequence of characters enclosed in double quotes. When string constants are used in C program, it automatically initializes a null at end of string.

Ex: "Hello" "Welcome" "Welcome to C Lab"

Reading strings from terminal

Formatted input function

scanf() can be used with %s format specification to read a string.

Example

```
char name[10];
scanf("%s",name);
```

Here don't use „&“ because name of string is a pointer to array. The problem with scanf() is that it terminates its input on the first white space it finds.

Example

```
#include <stdio.h>

int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

Output

Enter name: HelloWorld

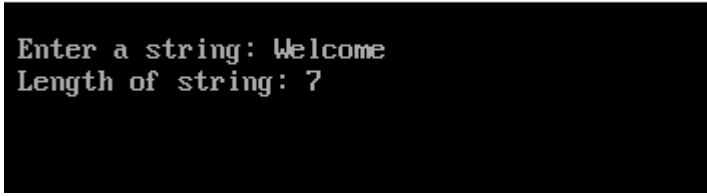
Your name is HelloWorld.

Program xx: Finding string length using scanf and printf

```
#include <stdio.h>
#include <conio.h>

int main()
{
    char s[1000], i;
    clrscr();
    printf(" \n Enter a string: ");
    scanf("%s", s);
    for(i = 0; s[i] != '\0'; ++i);
    printf(" Length of string: %d", i);
    getch();
    return 0;
}
```

Output

 DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr

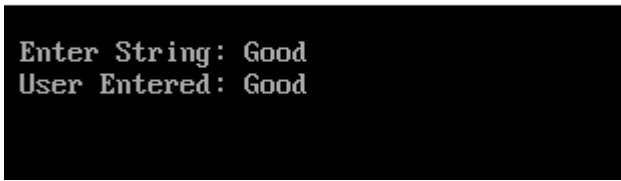
```
Enter a string: Welcome
Length of string: 7
```

Program xx: using unformatted input

```
#include<stdio.h>

int main()
{
    char name[30];
    printf("Enter String: ");
    gets(name);    //Function to read string from user.
    printf("User Entered: ");
    puts(name);    //Function to display string.
    return 0;
}
```

Output

 DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip

```
Enter String: Good
User Entered: Good
```

Unformatted input functions

getchar() It is used to read a single character from keyboard. Using this function repeatedly we may read entire line of text

Example

```
char ch='z';
ch=getchar();
```

gets() It is more convenient method of reading a string of text including blank spaces.

Example

```
char line[100];  
gets(line);
```

Writing strings on to the screen

Using formatted output functions:- printf with %s format specifier we can print strings in different formats on to screen.

Example

```
char name[10];  
printf("%s",name);
```

Using formatted output functions – printf() with %s format specifier we can print strings in different formats on to screen.

Example

```
char name[10];  
printf("%s", name);
```

Using unformatted output functions:

(a) **putchar()**: It is used to print a character on the screen.

Ex:- putchar(ch);

(b) **puts()**: It is used to print strings including blank spaces.

Ex:- char line[15]="Welcome to lab";

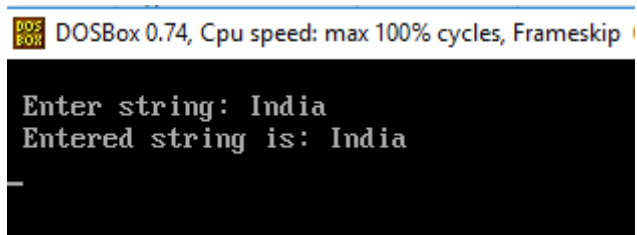
```
puts(line);
```

Program xx: using unformatted input

```
#include <stdio.h>  
#include <conio.h>  
int main()  
{  
    char name[30], ch;  
    int i = 0;  
    clrscr();  
    printf("\n Enter string: ");
```

```
while(ch != '\n') // terminates if user hit enter
{
    ch = getchar();
    name[i] = ch;
    i++;
}
name[i] = '\0'; // inserting null character at end
printf(" Entered string is: %s", name);
getch();
return 0;
}
```

Output

A screenshot of a DOSBox 0.74 window. The title bar reads 'DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip'. The terminal window has a black background with white text. It shows the prompt 'Enter string: India' followed by the user input 'India'. Below that, it shows 'Entered string is: India' followed by a blank line and a cursor.

String Functions

C supports a number of string handling functions. All of these built-in functions are aimed at performing various operations on strings and they are defined in the header file *string.h*

strlen()

This function is used to find the length of the string excluding the NULL character. In other words, this function is used to count the number of characters in a string. Its syntax is as follows:

```
int strlen(string);
```

for example

```
#include<stdio.h>
```

```
#include<string.h>
```

```
main()
```

```

{
    char string1[50];
    int length;
    printf("\n Enter any string:");
    gets(string1);
    length=strlen(string1);
    printf("\n The length of string=%d",length);
}

```

Output

Enter any string: Good

The length of string=4

strcpy()

This function is used to copy one string to the other. Its syntax is as follows:

```
strcpy(string1,string2);
```

where string1 and string2 are one-dimensional character arrays. This function copies the content of string2 to string1.

For example

```
#include<stdio.h>
```

```
#include<string.h>
```

```
main( )
```

```

{
    char string1[30],string2[30];
    printf("\n Enter first string:");
    gets(string1);
    printf("\n Enter second string:");
    gets(string2);
    strcpy(string1,string2);
    printf("\n First string=%s",string1);
    printf("\n Second string=%s",string2);
}

```

```
}
```

Output

```
Enter first string: Hello
```

```
Enter second string: Hi
```

```
First string= Hi
```

```
Second string= Hi
```

strcmp ()

This function compares two strings character by character (ASCII comparison) and returns one of three values {-1,0,1}. The numeric difference is “0” if strings are equal .If it is negative string1 is alphabetically above string2 .If it is positive string2 is alphabetically above string1. Its syntax is as follows:

```
int strcmp(string1,string2);
```

for example

```
#include<stdio.h>
```

```
#include<string.h>
```

```
main()
```

```
{
```

```
    char string1[30],string2[15];
```

```
    int x;
```

```
    printf(“\n Enter first string:”);
```

```
    gets(string1);
```

```
    printf(“\n Enter second string:”);
```

```
    gets(string2);
```

```
    x=strcmp(string1,string2);
```

```
    if(x==0)
```

```
        printf(“\n Both strings are equal”);
```

```
    else if(x>0)
```

```
        printf(“\n First string is bigger”);
```

```
    else
```

```

        printf("\n Second string is bigger");
    }

```

Output

```

Enter first string: Raja
Enter second string: Roja
Second string is bigger

```

strcat ()

This function is used to concatenate two strings. i.e., it appends one string at the end of the specified string. Its syntax as follows:

```
strcat(string1,string2);
```

where string1 and string2 are one-dimensional character arrays. This function joins two strings together. In other words, it adds the string2 to string1 and the string1 contains the final concatenated string. E.g., string1 contains prog and string2 contains ram, then string1 holds program after execution of the strcat() function.

For example

```

#include<stdio.h>
#include<string.h>
main()
{
    char string1[30],string2[15];
    printf("\n Enter first string:");
    gets(string1);
    printf("\n Enter second string:");
    gets(string2);
    strcat(string1,string2);
    printf("\n Concatenated string=%s",string1);
}

```

Output

Enter first string: Wel

Enter second string: Come

Concatenated string= WelCome